LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – INHERITANCE



NAMA : Elok Elysia Dewi

NIM : 103132400013

KELAS : S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Inheritance (Pewarisan)

Inheritance adalah salah satu pilar dasar OOP (Object Oriented Programming). Konsep ini memungkinkan sebuah kelas diturunkan dari kelas lain sehingga mewarisi ciri atau sifat kelas tersebut. Semakin ke bawah dalam hierarki, kelas semakin spesifik. Contohnya, kelas Pria dan Wanita mewarisi seluruh state dan behaviour dari kelas Manusia.

Hubungan antar kelas ini disebut hubungan "is a" (adalah bagian dari). Tujuan utamanya adalah menghindari duplikasi field, variabel, dan method yang sebenarnya bisa diturunkan dari satu kelas.

Ada dua istilah penting:

- Superclass (base class): kelas yang menurunkan.
- Subclass (derived class): kelas yang diturunkan. Subclass memiliki semua karakteristik superclass-nya, ditambah anggota tambahan miliknya sendiri.

Di Java, pewarisan dilakukan dengan keyword extends:

```
class B extends A { ... }
```

Artinya kelas B mewarisi atribut dan method kelas A, sehingga objek B bisa memanggil tampilXY() milik A, sementara anggota baru seperti z dan jumlahXY() hanya ada di B.

2. Keyword super

Keyword super dipakai untuk merujuk ke superclass dari suatu kelas, baik atribut maupun method-nya. Kegunaan utamanya:

- super(...) untuk memanggil konstruktor superclass.
- super.namaAtribut untuk mengakses atribut superclass.
- super.namaMethod() untuk memanggil method superclass.

Pada contoh Employee dan Manager, konstruktor Manager memanggil super(nama) untuk menginisialisasi atribut name milik Employee. Atribut name bersifat private, jadi tidak bisa diakses langsung dari subclass dan harus lewat konstruktor atau method milik superclass.

3. Overloading Method

Overloading adalah membuat dua atau lebih method dengan nama yang sama dalam satu kelas, tetapi jumlah atau tipe parameternya berbeda. Java menentukan method mana yang dipanggil berdasarkan argumen yang diberikan. Contohnya method IsiParam(String) dan IsiParam(String, String) pada kelas Lagu.

4. Overriding Method

Overriding terjadi ketika subclass mendefinisikan ulang method yang sudah ada di superclass dengan nama, parameter, dan tipe kembalian yang sama. Jika method tersebut dipanggil dari objek subclass, yang dijalankan adalah versi milik subclass, bukan superclass. Versi superclass tetap bisa dipanggil lewat super.namaMethod(), seperti pada contoh tampilkanKeLayar() di kelas A dan B.

5. **Perbedaan Overloading dan Overriding**

Aspek	Overloading	Overriding
Lokasi	Dalam satu kelas	Antara superclass dan subclass
Nama method	Sama	Sama
Parameter	Harus berbeda (jumlah/tipe)	Harus sama
Syarat	Tidak perlu pewarisan	Harus ada pewarisan

6. **Polymorphism**

   Polymorphism adalah kemampuan sebuah objek untuk mengambil beberapa bentuk berbeda. Dengan konsep ini, satu interface yang sama bisa dipakai untuk memerintah objek melakukan aksi yang prinsipnya sama tetapi prosesnya berbeda. Overriding adalah salah satu bentuk penerapannya di Java.

B. Guided

1. A (Inheritance)

```
package Guided1;


public class A {
    int x;
   int y;


   void tampilXY() {
       System.out.println("Nilai x: " + x + " nilai y: " + y);
   }
}
```

B

```
package Guided1;
```

```
public class B extends A {
    int z;
    int hasil;

    void jumlahXYZ() {
        hasil = x+y+z;

        System.out.println("jumlah (x + y + z) = " + hasil);
    }
}
```

Main

```
package Guided1;

public class Demo {
    public static void main(String[] args) {
        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();


        // member superclass dapat diakses dari subclassnya
```

```
        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;


        System.out.println("Subclass: ");
        subclass.jumlahXYZ();


    }
}
```

Output :

```
run:
Superclass:
Nilai x: 3 nilai y: 4
Subclass:
jumlah (x + y + z) = 8
BUILD SUCCESSFUL (total time: 0 seconds)
```

2. Employee (Keyword Super)

```
package Guided2;

public class Employee {
    private String name;
    String departemen;

    public Employee(String nama) {
        this.name = nama;
    }

    public void tampilNama() {
        System.out.println("nama : " + name);
    }
}
```

Manager

```
package Guided2;
```

```
public class Manager extends Employee {
    private String alamat;

    public Manager(String nama, String alamat) {
        /* memanggil konstruktor employee */
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi() {
        super.tampilNama();

        /* menampilkan variabel alamat dan departemen */
        System.out.println("alamat : " + alamat);
        System.out.println("departemen : " + super.departemen);
    }
}
```

DemoManager

```
package Guided2;

public class DemoManager {
    public static void main(String[] args) {
        /* membuat objek */
        Manager adi = new Manager("Adi", "Sukabirus");
        adi.departemen = "SDM";
        adi.tampilInformasi();
    }
}
```

Output :

```
run:
nama : Adi
alamat : Sukabirus
departemen : SDM
BUILD SUCCESSFUL (total time: 1 second)
```

3. Lagu (Overloading Method)

```
package Guided3;

public class Lagu {
```

```
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void Tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}
```

Main

```
package Guided3;

public class Main {
    public static void main(String[] args) {
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");

        d.Tampilkan();
        e.Tampilkan();
    }
}
```

Output:

```
run:
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : Lagu 2, pencipta : aku
BUILD SUCCESSFUL (total time: 2 seconds)
```

4. A (Overiding Method)

```
package Guided4;

public class A {

    public void tampilkan() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

B

```
package Guided4;

public class B extends A {

    public void tampilkan() {
        super.tampilkan(); // Method milik superclas dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}
```

Main

```
package Guided4;

public class Main {
    public static void main(String[] args) {
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output:

```
run:
Method milik class A dipanggil...
Method milik class B dipanggil...
BUILD SUCCESSFUL (total time: 2 seconds)
```

C. Unguided

Soal

Sebuah tim machine learning ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis deep learning yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Ketentuan

1. Kelas induk (model umum)
    - Menyimpan nama model (teks) dan jumlah data latih (bilangan bulat).
    - Memiliki constructor yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.
    - Memiliki method untuk menampilkan informasi.
    - Memiliki dua method untuk menghitung biaya komputasi dengan nama yang sama (overloading):
        - Tanpa parameter: biaya = jumlah data latih × 2.
        - Dengan satu parameter bilangan bulat (jumlah epoch): biaya = jumlah data latih × epoch.
2. Kelas turunan pertama (model regresi)
    - Mewarisi kelas induk dan menambahkan atribut jumlah fitur.
    - Constructor menerima nama, jumlah data latih, dan jumlah fitur; gunakan constructor kelas induk.
    - Override method tampilkan informasi dari kelas induk.
    - Override method hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur × 10).
    - Menambahkan satu method baru dengan nama yang sama dengan method penghitung biaya, tetapi menerima dua parameter bilangan bulat (jumlah epoch dan ukuran batch): biaya = jumlah data latih × epoch ÷ ukuran batch. Sehingga objek kelas ini memiliki tiga versi method penghitung biaya.
3. Kelas turunan kedua (model klasifikasi)
    - Mewarisi kelas induk dan menambahkan atribut jumlah kelas.
    - Constructor menerima nama, jumlah data latih, dan jumlah kelas; gunakan constructor kelas induk.
    - Override method tampilkan informasi dari kelas induk.
    - Override method hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) × jumlah kelas.
    - Versi penghitung biaya dengan parameter epoch tidak di-override.
4. Kelas turunan ketiga (model klasifikasi deep learning)

- Mewarisi kelas turunan kedua (bukan kelas induk langsung) dan menambahkan atribut jumlah layer.
- Constructor menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer; gunakan constructor kelas di atasnya.
- Override method tampilkan informasi dari kelas di atasnya.
- Override method hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer × 200).

5. Class Main
   - Buat tiga objek berikut:

Jenis model	Nama	Data latih	Atribut tambahan
Regresi	"Regresi Linier"	500	4 fitur
Klasifikasi	"Decision Tree"	800	3 kelas
Klasifikasi deep learning	"Neural Network"	1000	5 kelas, 3 layer

   - Simpan ketiga objek tersebut dalam satu array bertipe kelas induk.
   - Gunakan perulangan untuk setiap elemen array: tampilkan informasi model, tampilkan biaya komputasi, lalu jumlahkan seluruh biaya.
   - Setelah perulangan, tampilkan total biaya.
   - Terakhir, tampilkan juga:
     - biaya 10 epoch untuk model “Neural Network” (ambil dari array),
     - biaya 10 epoch untuk model “Regresi Linier”,
     - biaya 10 epoch dengan ukuran batch 50 untuk model “Regresi Linier”.

   Contoh Output

```
None
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Jawaban:

ModelML

```
package Unguided;

public class ModelML {
    private String nama;
    private int jumlahDataLatih;

    public ModelML(String nama, int jumlahDataLatih) {
        this.nama = nama;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + nama + " | Data latih: " + jumlahDataLatih);
    }

    // Overloading: tanpa parameter
    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }
```

```
    // Overloading: dengan parameter epoch
    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

Penjelasan:
Kelas induk ini menyimpan nama dan jumlahDataLatih yang bersifat private, diisi lewat constructor. Karena private, disediakan getter getJumlahDataLatih() supaya kelas turunan tetap bisa membacanya. Method tampilkanInfo() menampilkan nama dan data latih. Method hitungBiaya() dibuat dua versi (overloading), yaitu tanpa parameter (data latih × 2) dan dengan parameter epoch (data latih × epoch).

ModelRegresi

```
package Unguided;

public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    public ModelRegresi(String nama, int jumlahDataLatih, int jumlahFitur) {
        super(nama, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    // Override
    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
```

```
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    // Override
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // Overloading baru: epoch dan ukuran batch
    public int hitungBiaya(int epoch, int ukuranBatch) {
        return getJumlahDataLatih() * epoch / ukuranBatch;
    }
}
```

Penjelasan:
Kelas ini mewarisi ModelML dengan extends dan menambahkan atribut jumlahFitur. Constructor-nya memanggil super(...) untuk mengisi atribut induk. Method tampilkanInfo() dan hitungBiaya() di-override: info induk ditambah jumlah fitur, dan biaya induk ditambah (fitur × 10). Kelas ini juga punya hitungBiaya(int epoch, int ukuranBatch) sebagai overloading baru, sehingga total ada tiga versi hitungBiaya.

ModelKlasifikasi

```
package Unguided;


public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;
```

```
    public ModelKlasifikasi(String nama, int jumlahDataLatih, int jumlahKelas) {
        super(nama, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    // Override
    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    // Override
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }

    // hitungBiaya(int epoch) TIDAK di-override
}
```

Penjelasan:
Kelas ini juga mewarisi ModelML dan menambahkan atribut jumlahKelas. Method tampilkanInfo() di-override untuk menambah jumlah kelas, dan hitungBiaya() di-override sehingga biaya induk dikalikan jumlah kelas. Versi hitungBiaya(int epoch) tidak di-override, jadi memakai versi milik ModelML.

ModelKlasifikasiDL

```
 package Unguided;


public class ModelKlasifikasiDL extends ModelKlasifikasi {
    private int jumlahLayer;


    public ModelKlasifikasiDL(String nama, int jumlahDataLatih, int jumlahKelas,
int jumlahLayer) {
        super(nama, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }


    // Override
    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }


    // Override
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

Penjelasan :
Kelas ini mewarisi ModelKlasifikasi, bukan ModelML langsung, sehingga terjadi pewarisan bertingkat. Atribut barunya adalah jumlahLayer, dan super(...) di constructor-nya menunjuk ke ModelKlasifikasi. Method tampilkanInfo() dan hitungBiaya() di-override dengan memanggil versi kelas di atasnya, lalu menambah info layer dan biaya (layer × 200).

Main

```
package Unguided;


public class Main {
    public static void main(String[] args) {
        // Array bertipe kelas induk (polymorphism)
        ModelML[] daftarModel = {
            new ModelRegresi("Regresi Linier", 500, 4),
            new ModelKlasifikasi("Decision Tree", 800, 3),
            new ModelKlasifikasiDL("Neural Network", 1000, 5, 3)
        };

        int totalBiaya = 0;

        for (ModelML m : daftarModel) {
            m.tampilkanInfo();
            int biaya = m.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }
```

```
        System.out.println("Total biaya: " + totalBiaya);


        // Neural Network = indeks 2, Regresi Linier = indeks 0
        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
daftarModel[0].hitungBiaya(10));


        // Perlu casting karena hitungBiaya(int, int) hanya ada di ModelRegresi
        ModelRegresi regresi = (ModelRegresi) daftarModel[0];
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
regresi.hitungBiaya(10, 50));
    }
}
```

Penjelasan :
Tiga objek dibuat dan disimpan dalam satu array bertipe ModelML, yang bisa dilakukan karena ketiganya turunan ModelML (polymorphism). Perulangan for menampilkan info dan biaya tiap model, sambil menjumlahkannya ke totalBiaya. Method yang dijalankan mengikuti tipe objek aslinya. Setelah itu dicetak total biaya, lalu biaya 10 epoch untuk Neural Network dan Regresi Linier. Untuk biaya dengan batch, objek regresi harus di-casting ke ModelRegresi karena method dua parameter itu hanya ada di kelas tersebut.

Output :

```
run:
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
BUILD SUCCESSFUL (total time: 0 seconds)
```

Penjelasan :

Program menampilkan informasi dan biaya komputasi untuk tiga model sesuai urutan array. Model pertama, Regresi Linier dengan 500 data latih, menampilkan jumlah fitur 4 dan biaya 1040. Angka ini berasal dari (500 × 2) + (4 × 10), yaitu biaya dasar 1000 ditambah biaya fitur 40.

Model kedua, Decision Tree dengan 800 data latih, menampilkan jumlah kelas 3 dan biaya 4800. Biaya dasarnya 1600 (800 × 2), lalu dikalikan jumlah kelas 3.

Model ketiga, Neural Network dengan 1000 data latih, menampilkan jumlah kelas 5 dan jumlah layer 3, dengan biaya 10600. Biaya dasarnya 2000 dikalikan 5 kelas menjadi 10000, lalu ditambah 600 dari 3 layer × 200. Info yang tampil lebih banyak karena kelas ini mewarisi ModelKlasifikasi, sehingga menampilkan info induknya dulu baru jumlah layer.

Setiap model dipisahkan garis ---. Setelah semua model tampil, program mencetak total biaya 16440, hasil dari 1040 + 4800 + 10600.

Tiga baris terakhir menunjukkan overloading. Biaya 10 epoch untuk Neural Network adalah 10000 (1000 × 10) dan untuk Regresi Linier 5000 (500 × 10), keduanya memakai method hitungBiaya(int epoch) milik ModelML. Biaya 10 epoch dengan batch 50 untuk Regresi Linier adalah 100, dari (500 × 10) ÷ 50, yang memakai method dua parameter milik ModelRegresi.

D. Kesimpulan

1. **Inheritance** memungkinkan sebuah kelas (subclass) mewarisi atribut dan method dari kelas lain (superclass) dengan keyword extends. Dengan begitu kode tidak perlu ditulis berulang dan hierarki kelas menjadi lebih terstruktur dari yang umum ke yang spesifik.
2. **Keyword super** digunakan untuk mengakses constructor, atribut, dan method milik superclass, sehingga subclass bisa memanfaatkan dan melengkapi perilaku superclass-nya.
3. **Overloading** adalah pembuatan beberapa method dengan nama sama dalam satu kelas yang dibedakan oleh jumlah atau tipe parameternya.
4. **Overriding** adalah pendefinisian ulang method superclass di subclass, sehingga method yang dijalankan adalah milik subclass.
5. **Polymorphism** membuat objek dari kelas turunan yang berbeda dapat diperlakukan sebagai tipe superclass-nya, dan method yang dijalankan menyesuaikan tipe objek sebenarnya.